单卡 H800 上的 LingBot-VLA 2.0 部分参数微调：RoboTwin 2.0 挑战赛工程复盘

BotLab；Claude Opus 5.5

2026 年 10 月

目录

摘要 **2**

1 任务与约束 **2**
1.1 赛题 . . . 2
1.2 我们的约束 . . . 2

2 训练方案 **2**
2.1 训练范围的选择 . . . 2
2.2 训练配置 . . . 3
2.3 训练过程中的稳定性问题 . . . 3
2.4 训练产物的搬运 . . . 3

3 评测工程 **3**
3.1 单卡多客户端 . . . 3
3.2 评测卡死 . . . 3
3.3 统计口径 . . . 4
3.4 重跑的随机性 . . . 4

4 推理参数扫描 **4**

5 结果 **4**
5.1 训练过程中的快照 . . . 4
5.2 正式自测 . . . 5
5.3 失败分析 . . . 6

6 提交与复现演练 **6**
6.1 打包与上传 . . . 6
6.2 评委视角复现演练 . . . 6
6.3 中文路径与 ZIP64 的解压问题 . . . 7
6.4 补交 . . . 7

7 局限与反思 **7**

附录 A 复现 **8**

附录 B 费用（估算，以平台账单为准） **8**

致谢 **8**

摘要

本文复盘 BotLab 参加蚂蚁灵波具身大模型挑战赛初赛的完整过程。任务是基于 `robbyant/lingbot-vla-v2-6b`，只用 RoboTwin 2.0 Aloha-AgileX 50 个任务的 clean 示范数据（每任务 50 条）训练一份权重，并在 clean 与 randomized 两种设定下评测。受算力限制，我们没有做全量微调，而是在单卡 H800 上训练 “VLM 注意力层 LoRA + 非 VLM 参数全量”（可训练参数占 29.0%），共 20000 步、约 58 小时，全程花费约 ¥800（估算）。

最终权重在每任务 30 次的自测中，clean 成功率 50.6%（759/1500），randomized 25.5%（383/1500）。

本文的主要内容不在成绩，而在过程中积累的工程经验：

- 长时间单卡训练的稳定性问题，以及两阶段恢复；
- 单卡多客户端评测（7 个客户端时吞吐约 3.5 倍）与显存容量估计；
- 评测卡死的定位方法；
- 推理参数扫描的配对统计方法，以及 “参数基本不敏感” 的负结果；
- 第一次提交后的评委视角复现演练：在干净机器上只凭提交包从零复现。演练与自测的逐 seed 一致率为 77.3%，在同权重重跑的正常波动范围内；从零准备的训练数据与原始版本逐字节一致。演练还发现了几处文档缺失和一个跨平台解压问题，据此补交了一次。

1 任务与约束

1.1 赛题

- 模型起点：`robbyant/lingbot-vla-v2-6b`（VLM 约 4.4B + MoE 动作专家约 1.8B）。不允许用官方 `lingbot-vla-v2-6b-robotwin` 作起点或教师。
- 训练数据：RoboTwin 2.0 Aloha-AgileX 50 个任务的 `demo_clean`，每任务 50 条；randomized 数据、unseen 指令、外部数据、仿真额外 rollout 一律禁止。
- 评测：clean（seen 指令）与 randomized（unseen 指令、随机背景与杂物）两种设定，各 50 个任务，标准为每任务 100 次；推理只能用一份权重，不能按任务路由。

1.2 我们的约束

- 队伍 2 人，算力自费（AutoDL 按小时租卡），初赛周期约两周。
- AMD 赛事算力预计 11 月初才发放，晚于初赛截止，因此全程使用 AutoDL：训练用 H800 PCIe 80 GB（¥9.98/时），评测用 RTX 4080 SUPER 32 GB（¥1.58/时）。

2 训练方案

2.1 训练范围的选择

开训前我们实测了三种方案的速度（global batch 128）：

方案	可训练部分	卡数	速度	每千步成本
A	只训动作专家	1	7.3 s/step	约 ¥20
B	全模型（官方配方）	2	28.8 s/step	约 ¥160
C	VLM 注意力 LoRA（r32/α64）+ 非 VLM 参数全参	1	9.6 s/step	约 ¥27

全模型 fp32 加 Adam 状态约 100 GB，单张 80 GB 卡放不下；租到的双卡机器之间没有 P2P，双卡几乎没有加速。按预算内能训的步数，选择 C 训练 20000 步。C 需要一个新的 LoRA 作用范围，我们以补丁形式加入上游代码（`lora_scope=vlm_expert_full`）。

2.2 训练配置

- 优化：AdamW，lr 1e-4，余弦衰减到 5e-5，global batch 128（micro batch 16），seed 42。
- 精度与编译：fp32/TF32（与官方配方一致），开启 `torch.compile`。不编译时 flex attention 走逐元素回退实现，每步慢 3–5 倍。
- 数据处理（均在赛规允许范围内）：
  - 归一化统计量只用 50 个 clean 数据集重算（54,978 帧）；官方 `robotwin.json` 有 6,062,592 帧，远多于 clean 数据的帧数，我们据此推断它包含 randomized 数据，没有使用。
  - 训练时把任务文本随机替换为同一条轨迹自带的另一条 seen 指令（“seen 池内替换”），以缓解训练只见每条轨迹第一条指令、评测却随机抽取指令的分布差异。
  - 框架自带的颜色增广（亮度、对比度、饱和度）。
- 不使用教师模型（官方配方中的深度、视频对齐教师）。

2.3 训练过程中的稳定性问题

单卡连续训练近 60 小时，遇到的主要问题及处理：

1. **容器内存看门狗**：AutoDL 容器内存上限 120 GiB。训练进程两次在大量文件 I/O 期间被 SIGKILL，cgroup 没有记录 OOM。`memory.current` 把页缓存也算在内；我们推测（未证实）平台看护进程按它判断超限，DataLoader 预取占用的共享内存和保存 checkpoint 写出的约 37 GB 页缓存叠加，就会越过上限。处理：降低预取深度（4→2）；每 15 秒对输出文件执行 `fdatasync` + `posix_fadvise(DONTNEED)` 释放页缓存（单次清理从 110 GiB 降到 87 GiB；两项合起来，保存窗口的内存峰值从 118 GiB 降到 90 GiB）；LoRA 合并改为逐组流式处理（与 AMD 参考样例的合并结果逐张量比对一致）。
2. **checkpoint 轮转**：只保留 1 个 checkpoint 时，上游逻辑先删旧、再写新。第 8000 步保存中途进程被杀，结果两头落空。处理：保留 2 个 checkpoint，每次启动前把不完整的 checkpoint 移到隔离目录。
3. **两阶段恢复**：由于第 2 点，第 7500 步之后没有可续训的 checkpoint。我们把第 7500 步的 LoRA 合并权重作为第二阶段起点，lr 取第一阶段在 7500 步时的值（8.46e-5），2% 预热后余弦衰减，再训 12500 步，合计 20000 步。优化器状态因此重置一次。复现脚本用 “按 20k 余弦计划、在 7500 步保存退出” 的开关还原这一过程，但多次中断续训的数据顺序无法逐位复现。

2.4 训练产物的搬运

训练机（西北）到评测机（北京）跨区单流传输约 1 MB/s，每个快照 12 GB。我们按文件并行传输（约 11 MiB/s），传完后逐文件比对 sha256，再在评测机上改写快照内配置文件的绝对路径。整个流程用一个循环脚本自动完成，训练期间每 2500 步产出一个快照供中途评测。

3 评测工程

3.1 单卡多客户端

RoboTwin 每个评测进程主要耗时在仿真上（单核 CPU 满载），推理只占每个 episode 墙钟时间的约 10%；原先一张 4080 只能放两个推理服务（各约 12.5 GB）。我们确认了推理服务在 chunk 模式（每次调用返回整段动作）下对调用之间无状态、请求串行处理，于是让一个推理服务同时服务多个评测客户端。每个任务仍是独立的 `--seed 0` 运行，seed 与 episode 的对应关系不变。

- 吞吐：7 个客户端时约为原方案的 3.5–3.8 倍（实测，每任务 1 次的评测轮次）。
- 显存是唯一的约束：每个客户端渲染占用的显存会随场景波动，clean 场景为 2.1–3.1 GB，randomized 场景（随机背景与杂物）最高 5.5 GB。8 个客户端时总显存打到上限，出现渲染失败，被记为 episode 失败，该次运行整体作废。最终上限定为 clean 6 个、randomized 3 个，写在配置文件里，由评测脚本自动截断。
- 我们按历史耗时把长任务排在前面先跑，减少多客户端在收尾阶段的空等。

3.2 评测卡死

评测中偶有任务长时间没有任何输出。我们给评测脚本加了看门狗：任务日志长时间不更新时，先对进程组发 SIGABRT（Python faulthandler 会把各线程调用栈写入日志），再结束进程，并从第一个 seed 起整任务重跑。

抓到的调用栈显示，卡在专家检查阶段的 MPLib 运动规划（`IK` / `plan_qpos`），发生在策略执行之前，与模型和推理服务无关；IK 带随机初值，所以是非确定性的。看门狗加入之前，clean 有两个任务卡住数小时，当时没有调用栈，只看到客户端停在等待推理服务回复，原因未确认。

一个教训是，阈值最初设为 20 分钟，误杀了专家规划本来就慢的任务（如 randomized 的 `move_can_pot`，单个 episode 约 22 分钟）。改为 60 分钟后再也没有误杀，真正的卡死都持续数小时，仍然能被捕获。

3.3 统计口径

本次使用的评测脚本把初始状态不稳定、未执行策略的场景也计入测试次数。提交的 results.json 直接取脚本输出；如只统计实际执行的 episode，clean 为 52.97%，randomized 为 30.21%。

3.4 重跑的随机性

同一权重、同样 `--seed 0` 重跑，逐 episode 的成败并不完全一致。原因有两个：策略的 flow matching 采样带随机噪声；专家检查用的 MPLib IK 初值是随机的。

我们用推理参数扫描中的几组运行（同一权重，参数差异都在误差内）与正式自测按 (任务, seed) 配对，逐 seed 一致率为 70–77%。第一次提交后的复现演练（第 6.2 节）换了一台机器、重新安装环境，一致率为 77.3%。

因此，每任务 30 次时，单任务成功率的波动很大，只有汇总成功率才有比较意义。

4 推理参数扫描

推理阶段在单一权重下可以调整的参数主要有动作块执行长度 `USE_LENGTH`（每次推理生成 50 步，执行其中前若干步）和去噪步数（默认 10）。我们在 16 个任务的固定子集上（按 clean 成功率分四档，每档用固定种子抽 4 个）比较各参数，每个任务 10 次。

为减少噪声，比较时只取两边都实际执行的 episode，并按 (任务, seed) 配对；置信区间以任务为单位做 bootstrap (2000 次)，另做 “去掉贡献最大的任务” 的稳健性检查。事先定下的采用规则：clean 与 randomized 方向一致、95% 置信区间下限大于 0、去掉贡献最大的任务后仍为正，三条同时满足。

变体（相对默认 25 / 10 步）	clean 配对差值（95% CI）	randomized 配对差值（95% CI）
`USE_LENGTH` 10	前 3 个任务 2/30 对 12/30，提前停止	未测
`USE_LENGTH` 35	+5.4%（−5.8%, +16.2%）	−4.5%（−13.5%, +4.8%）
`USE_LENGTH` 50	+4.1%（−6.1%, +15.4%）	未测（clean 与 35 相当）
去噪 5 步	−0.7%（−6.5%, +4.1%）	未测
去噪 20 步	+2.9%（−5.5%, +10.7%）	未测

结论：除执行长度过短明显变差外，其余参数都在噪声范围内，最终保持默认值。单个任务对执行长度很敏感，例如 `press_stapler` 在 clean 下从 3/10 升到 8–9/10，`place_empty_cup` 在 randomized 下从 9/10 降到 5/10，方向因任务而异；赛规不允许按任务选择参数，全局统一取值后，这些差异相互抵消。扫描使用了评测场景，存在一定选择偏差。

5 结果

5.1 训练过程中的快照

全局步	clean	randomized	每任务次数
7500	32%	32%	1
15000	43%	34%	2
19000	52%	30%	1

全局步	clean	randomized	每任务次数
20000	48%	32%	1
20000（正式自测）	50.6%	25.5%	30

clean 随训练持续上升，randomized 在 30% 左右基本不动。前四行每任务只测 1–2 次，误差约 ±14 个百分点，只能看趋势。

5.2 正式自测

全局 20000 步，默认推理参数，每任务 30 次，评测环境为 RTX 4080 SUPER + CUDA + MPLib。下表为逐任务成功次数（“实跑” 为实际执行的 episode 数；表内按 clean 成功次数排序）：

任务	clean 成功/30	clean 实跑	randomized 成功/30	randomized 实跑
grab_roller	30	30	16	27
place_empty_cup	28	30	16	28
click_bell	26	30	27	29
stack_bowls_two	26	30	10	24
beat_block_hammer	25	30	6	26
place_container_plate	25	30	14	26
pick_dual_bottles	24	30	6	28
place_burger_fries	24	30	19	26
place_object_stand	24	30	16	26
move_playingcard_away	23	30	5	26
open_laptop	23	30	6	28
handover_mic	22	30	1	28
click_alarmclock	21	30	16	29
place_bread_basket	21	30	11	29
stack_blocks_two	21	30	12	27
place_object_basket	20	30	7	22
stack_bowls_three	19	30	4	22
adjust_bottle	18	19	11	22
dump_bin_bigbin	18	18	4	12
place_a2b_left	18	30	6	27
rotate_qrcode	17	30	5	27
shake_bottle	17	17	18	21
place_bread_skillet	16	30	5	23
shake_bottle_horizontally	16	17	16	21
place_a2b_right	15	30	8	28
place_phone_stand	15	30	7	23
turn_switch	15	30	3	26
move_pillbottle_pad	14	30	4	26
place_shoe	14	30	5	24
blocks_ranking_size	13	30	2	27
place_object_scale	12	30	6	25
press_stapler	12	30	24	28
move_can_pot	11	30	1	26
open_microwave	11	30	5	25
pick_diverse_bottles	11	15	4	14
place_cans_plasticbox	11	30	12	28
blocks_ranking_rgb	10	30	5	29
place_can_basket	9	30	2	26
place_fan	9	30	2	22
put_bottles_dustbin	9	30	10	27

任务	clean 成功/30	clean 实跑	randomized 成功/30	randomized 实跑
hanging_mug	8	30	0	26
lift_pot	8	30	0	25
scan_object	8	30	8	28
place_dual_shoes	7	27	1	25
put_object_cabinet	6	30	4	28
place_mouse_pad	3	30	4	27
stamp_seal	3	30	0	23
handover_block	2	30	4	25
move_stapler_pad	1	30	3	26
stack_blocks_three	0	30	2	27
合计	**759/1500**	1433	**383/1500**	1268

5.3 失败分析

- **多步任务**表现很差：stack_blocks_three（clean 0/30）、handover_block（2/30）、hanging_mug（8/30），randomized 下基本为 0。
- **精细定位任务**很弱：move_stapler_pad（1/30）、stamp_seal（3/30）、place_mouse_pad（3/30）。
- **clean 到 randomized 的落差**集中在需要识别和抓取特定物体的任务，例如 handover_mic 22→1、pick_dual_bottles 24→6、open_laptop 23→6、move_playingcard_away 23→5。按钮、摇瓶这类目标显著、动作简单的任务几乎不受影响（click_bell 26→27、shake_bottle 17→18）。我们推测这与 VLM 只做了 LoRA 微调、视觉表示对随机背景与杂物的适应不足有关，未做验证。

6 提交与复现演练

6.1 打包与上传

提交包是一个约 12.5 GB 的 zip，含 7 个权重分片。打包和上传遇到两个本地条件的限制：

- **本地磁盘**：本地磁盘空间不足。第一次打包时把权重复制了一份，又在同一块盘上写 zip，结果写满了磁盘。改进做法：权重分片用硬链接放进包目录，zip 写到另一块空间充足的盘。打包脚本在写完后回读 CRC，并从 zip 内流式计算每个权重文件的 sha256，与清单比对，不必再解压一次。
- **上行带宽**：本地上行带宽低，12.5 GB 传完所需的时间远超上传凭证 1 小时的有效期。我们改从评测机上传：
  - 评测机上已有同样的权重分片，只需从本地传过去 15 MB 的小文件和条目元数据；
  - 用同版本 Python 的 zipfile、同样的条目顺序和时间戳，在评测机上重建出 sha256 与本地完全相同的 zip；
  - 再用 ossutil 上传，约 14 MB/s，15 分钟传完。

6.2 评委视角复现演练

赛事《常见 Q&A》中 “初赛的 Review & 选拔逻辑” 一条写明，入围决赛的团队 “从训练到推理都会复现”，复现细节缺失导致无法复现的后果由参赛团队承担。为此，第一次提交后，我们模拟评委，只用提交的 zip 和包内 README，在一台全新的 AutoDL 实例上（干净镜像，RTX 4080 SUPER）从零走了一遍：

阶段	结果
取包与解压	zip 的 sha256 与提交的文件一致；权重清单 23 项中 21 项一致，另外两个 yaml 的路径打包时已改写，属预期
安装环境	setup_env.sh 79 分钟、fetch_assets.sh 70 分钟，全程无人工干预；GPU 自检和渲染测试通过
评测	两种设定各 50 任务 × 2 次，eval.sh 一次跑通。与自测按 seed 配对，一致率 clean 77.3%（97 对）、randomized 77.3%（88 对），在同权重重跑的范围内（3.4 节）；指令类型正确；results.json 通过官方模板校验

阶段	结果
训练准备	从零下载并转换数据：训练清单、seen 指令池与原始训练所用的文件逐字节一致；clean 统计量的帧数同为 54,978，数值最大相对差 2.6×10^{-6}

训练本身（H800 约 58 小时）没有复跑。演练没有发现会导致复现失败的问题，但发现了几处文档缺失：

- 没写基础镜像，操作手册里的 CUDA 版本也写错了；
- 没写数据准备的内存要求。在 2 GB 内存的无卡模式下，转换进程会被系统杀掉；
- 没写重跑结果的随机性；
- 没写下一节的解压问题。

6.3 中文路径与 ZIP64 的解压问题

在演练机上用 `unzip` 解压时，后半部分条目被解到一个 `#U521d#U8d5b...` 形式的乱码目录里，`unzip` 退出码为 1，文件内容本身完好。排查结果：

- **直接原因**：AutoDL 镜像设置了 `LANG=en_US.UTF-8`，但没有安装这个 locale，实际按 C locale 运行。
- **触发条件**（本地用小样本实验确定）：条目**同时**带 UTF-8 文件名标志和 ZIP64 扩展字段时，Debian 的 unzip 会把中文路径转义成 `#U` 形式；只具备其中一个条件的条目都正常。
- **影响范围**：Python 的 zipfile 在条目偏移超过 2 GiB 时写入 ZIP64 字段，所以 2 GiB 之后的条目都受影响。Info-ZIP 的 `zip -fz` 打出的包同样有这个问题。
- **无法根治**：去掉 UTF-8 标志可以绕开，但中文 Windows 的资源管理器会把文件名解成乱码（实测）。目录名又必须用赛事模板规定的中文。所以这个问题在打包层面无法根治。

最终的缓解办法：

- zip 条目改为小文件在前、权重分片在后，让 README、报告、results.json 和权重配置都落在前 14 MB；
- README 写明 `LANG=C.UTF-8 unzip` 和 `python3 -m zipfile -e` 两种解压方式。

在演练机上实测：直接 `unzip` 时只有第 3–7 个分片落进乱码目录，README 能正常读到；用这两种方式解压，137 个文件全部正确。Windows 资源管理器和 PowerShell 用小样本实测，结果正确。

6.4 补交

演练后，我们按评委视角补全了 README。补充内容包括：

- 基础镜像；
- 运行环境：操作系统、GPU、AMD（未实测，如实说明）、磁盘、网络；
- 解压与权重校验；
- 评测耗时与并行上限；
- 重跑的随机性。

然后用剩余 2 次中的 1 次补交（2026-10-25）。新包的权重和 results.json 与第一次逐字节相同，所以无论官方按哪次提交计，分数都不受影响；唯一的风险是新包本身出错，这一点用与第一次相同的整套校验来控制。

7 局限与反思

1. **没有全量微调**。我们推测这是成绩的主要瓶颈，未做验证。回头看，如果一开始就把预算集中在双卡全量训练上，并接受更少的步数，结果可能更好；我们在实际选型时只比较了速度，没有对不同训练范围做小规模的效果对比。
2. **评测投入偏多**。中途评测、参数扫描和正式自测合计约 70 小时 4080 时间。推理参数扫描最终是负结果，但它明确排除了“调推理参数能弥补训练不足”这条路。
3. **自测次数不足 100**。受时间限制，正式自测为每任务 30 次，单任务的统计误差仍然较大（95% 区间约 ±18 个百分点），总成功率约 ±2.5 个百分点。
4. **训练不可逐位复现**。两阶段恢复与多次中断续训使数据顺序与一次跑完不同；提交材料给出的是同配置的复现流程。

5. **复现演练做得太晚**。评委视角的演练是在第一次提交之后才做的，发现的文档问题只能靠补交修正，用掉了一次提交机会。如果在打包前就在干净机器上走一遍，可以一次提交到位。

附录 A 复现

```
# 解压（locale 缺失的 Linux 上不要直接 unzip，见 6.3 节）
LANG=C.UTF-8 unzip BotLab_ 初赛提交材料.zip
# 环境与资产（可在无卡模式下运行）
bash scripts/autodl/setup_env.sh && bash scripts/autodl/fetch_assets.sh
# 训练（AutoDL 单卡 H800 80 GB，数据盘挂在 /root/autodl-tmp）
bash train.sh
# 评测（各 50 任务 × 100 次）。单卡 32 GB 上客户端上限：
# clean 6 个、randomized 3 个，超出自动截断
CLIENTS=6 bash eval.sh <checkpoint_dir> both 100 0
```

代码仓库包含全部补丁（src/patches/）、配置（configs/）、测试（tests/）与决策日志（docs/decisions.md）。

附录 B 费用（估算，以平台账单为准）

项目	时长	费用
前期环境、数据准备与测速（4080 / H800 / 双卡 H800）	约 30 h	约 ¥46
H800 主训练（两阶段，20000 步）	约 58 h	约 ¥580
4080 评测（快照评测、参数扫描、正式自测）	约 70 h	约 ¥111
复现演练（无卡约 13 h + GPU 约 12 h）与补交	约 25 h	约 ¥24
数据盘（两台实例，至 10-25）	约 12 天	约 ¥40
合计		**约 ¥800**

致谢

感谢蚂蚁灵波与赛事主办方提供基座模型、数据与评测框架,感谢赛事介绍列出的 AMD 代码参考样例(Robotwin-radeon-cloud)，感谢 RoboTwin 2.0 团队开源仿真平台。